

LORA: LOW-RANK ADAPTATION OF LARGE LANGUAGE MODELS

Abstract—Edward Hu¹, Yelong Shen², Phillip Wallis³, Zeyuan Allen-Zhu⁴, Yuanzhi Li⁵, Shean Wang⁶, Lu Wang⁷, Weizhu Chen⁸ {yeshe, ph-wallis, zeyuana, swang, luw, wzchen}@microsoft.com, yuanzhil@andrew.cmu.edu An important paradigm of natural language processing consists of large-scale pre-training on general domain data and adaptation to particular tasks or domains. As we pre-train larger models, full fine-tuning, which re-trains all model parameters, becomes less feasible. Using GPT-3 175B as an example – deploying independent instances of fine-tuned models, each with 175B parameters, is prohibitively expensive. We propose Low-Rank Adaptation, or LoRA, which freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly reducing the number of trainable parameters for downstream tasks. Compared to GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by a factor of 10,000 and the GPU memory requirement by a factor of 3. LoRA performs on-par or better than fine-tuning in model quality on RoBERTa, DeBERTa, GPT-2, and GPT-3, despite having fewer trainable parameters, a higher training throughput, and, unlike adapters, no additional inference latency. We also provide an empirical investigation into rank-deficiency in language model adaptation, which sheds light on the efficacy of LoRA. We release a package that facilitates the integration of LoRA with PyTorch models and provide our implementations and model checkpoints for RoBERTa, DeBERTa, and GPT-2 at <https://github.com/microsoft/LoRA>.

I. INTRODUCTION

Many applications in natural language processing rely on adapting one large-scale, pre-trained language model to multiple downstream applications. Such adaptation is usually done via fine-tuning, which updates all the parameters of the pre-trained model. The major downside of fine-tuning is that the new model contains as many parameters as in the original model. As larger models are trained every few months, this changes from a mere “inconvenience” for GPT-2 (Radford et al., 2019) or RoBERTa large (Liu et al., 2019) to a critical deployment challenge for GPT-3 (Brown et al., 2020) with 175 billion trainable parameters.

Many sought to mitigate this by adapting only some parameters or learning external modules for new tasks. This way, we only need to store and load a small number of task-specific parameters in addition to the pre-trained model for each task, greatly boosting the operational efficiency when deployed. However, existing techniques

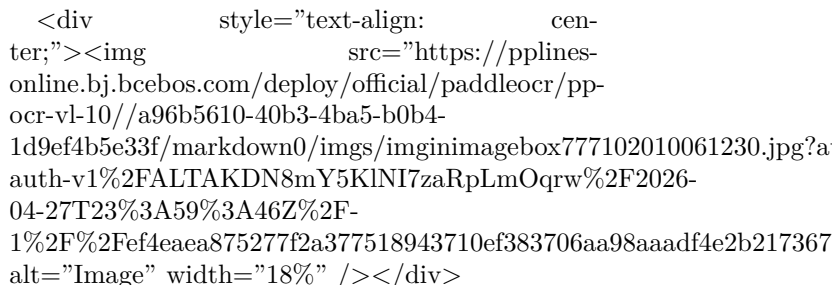


Figure 1: Our reparametrization. We only train A and B.

often introduce inference latency (Houlsby et al., 2019; Rebuffi et al., 2017) by extending model depth or reduce the model’s usable sequence length (Li & Liang, 2021; Lester et al., 2021; Ham

bardzumyan et al., 2020; Liu et al., 2021) (Section 3). More importantly, these methods often fail to match the fine-tuning baselines, posing a trade-off between efficiency and model quality.

We take inspiration from Li et al. (2018a); Aghajanyan et al. (2020) which show that the learned over-parametrized models in fact reside on a low intrinsic dimension. We hypothesize that the change in weights during model adaptation also has a low “intrinsic rank”, leading to our proposed Low-Rank Adaptation (LoRA) approach. LoRA allows us to train some dense layers in a neural network indirectly by optimizing rank decomposition matrices of the dense layers’ change during adaptation instead, while keeping the pre-trained weights frozen, as shown in Figure 1. Using GPT-3 175B as an example, we show that a very low rank (i.e., r in Figure 1 can be one or two) suffices even when the full rank (i.e., d) is as high as 12,288, making LoRA both storage- and compute-efficient.

LoRA possesses several key advantages.

- A pre-trained model can be shared and used to build many small LoRA modules for different tasks. We can freeze the shared model and efficiently switch tasks by replacing the matrices A and B in Figure 1, reducing the storage requirement and task-switching overhead significantly.

- LoRA makes training more efficient and lowers the hardware barrier to entry by up to 3 times when using adaptive optimizers since we do not need to calculate the gradients or maintain the optimizer states for most parameters. Instead, we only optimize the injected, much smaller low-rank

matrices.

- Our simple linear design allows us to merge the trainable matrices with the frozen weights when deployed, introducing no inference latency compared to a fully fine-tuned model, by construction.
- LoRA is orthogonal to many prior methods and can be combined with many of them, such as prefix-tuning. We provide an example in Appendix H.

Terminologies and Conventions We make frequent references to the Transformer architecture and use the conventional terminologies for its dimensions. We call the input and output dimension size of a Transformer layer d_{model} . We use W_{q} , W_{k} , W_{v} , and W_{o} to refer to the query/key/value/output projection matrices in the self-attention module. W or W_0 refers to a pretrained weight matrix and ΔW its accumulated gradient update during adaptation. We use r to denote the rank of a LoRA module. We follow the conventions set out by (Vaswani et al., 2017; Brown et al., 2020) and use Adam (Loshchilov & Hutter, 2019; Kingma & Ba, 2017) for model optimization and use a Transformer MLP feedforward dimension $d_{\text{ffn}} = 4 \times d_{\text{model}}$.

II. PROBLEM STATEMENT

While our proposal is agnostic to training objective, we focus on conditional language modeling as our motivating use case. Below is a brief description of the language modeling problem and, in particular, the maximization of conditional probabilities given a task-specific prompt.

Suppose we are given a pre-trained autoregressive language model $P_{\Phi}(y|x)$ parametrized by Φ . For instance, $P_{\Phi}(y|x)$ can be a generic multi-task learner such as GPT (Radford et al., 2019; Brown et al., 2020) based on the Transformer architecture (Vaswani et al., 2017). Consider adapting this pre-trained model to downstream conditional text generation tasks, such as summarization, machine reading comprehension (MRC), and natural language to SQL (NL2SQL). Each downstream task is represented by a training dataset of context-target pairs: $\{(x_i, y_i)\}_{i=1, \dots, |\mathcal{Z}|}$, where both x_i and y_i are sequences of tokens. For example, in NL2SQL, x_i is a natural language query and y_i its corresponding SQL command; for summarization, x_i is the content of an article and y_i its summary.

During full fine-tuning, the model is initialized to pre-trained weights Φ_0 and updated to $\Phi_0 + \Delta\Phi$ by repeatedly following the gradient to maximize the conditional language modeling objective:

$$\max_{\Phi} \sum_{(x,y) \in \mathcal{Z}} \log P_{\Phi}(y|x)$$

One of the main drawbacks for full fine-tuning is that for each downstream task, we learn a different set of parameters $\Delta\Phi$ whose dimension $|\Delta\Phi|$ equals $|\Phi_0|$. Thus, if the pre-trained model is large (such as GPT-3 with $|\Phi_0| \approx 175$ Billion), storing and deploying many independent instances of fine-tuned models can be challenging, if at all feasible.

In this paper, we adopt a more parameter-efficient approach, where the task-specific parameter increment $\Delta\Phi = \Delta\Phi(\Theta)$ is further encoded by a much smaller-sized set of parameters Θ with $|\Theta| \ll |\Phi_0|$. The task of finding $\Delta\Phi$ thus becomes optimizing over Θ :

$$\max_{\Theta} \sum_{(x,y) \in \mathcal{Z}} \log P_{\Phi_0 + \Delta\Phi(\Theta)}(y|x)$$

In the subsequent sections, we propose to use a low-rank representation to encode $\Delta\Phi$ that is both compute- and memory-efficient. When the pre-trained model is GPT-3 175B, the number of trainable parameters $|\Theta|$ can be as small as 0.01% of $|\Phi_0|$.

III. AREN'T EXISTING SOLUTIONS GOOD ENOUGH?

The problem we set out to tackle is by no means new. Since the inception of transfer learning, dozens of works have sought to make model adaptation more parameter- and compute-efficient. See Section 6 for a survey of some of the well-known works. Using language modeling as an example, there are two prominent strategies when it comes to efficient adaptations: adding adapter layers (Houlsby et al., 2019; Rebuffi et al., 2017; Pfeiffer et al., 2021; Rücklé et al., 2020) or optimizing some forms of the input layer activations (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). However, both strategies have their limitations, especially in a large-scale and latency-sensitive production scenario.

Adapter Layers Introduce Inference Latency There are many variants of adapters. We focus on the original design by Houlsby et al. (2019) which has two adapter layers per Transformer block and a more recent one by Lin et al. (2020) which has only one per block but with an additional LayerNorm (Ba et al., 2016). While one can reduce the overall latency by pruning layers or exploiting multi-task settings (Rücklé et al., 2020; Pfeiffer et al., 2021), there is no direct way to bypass the extra compute in adapter layers. This seems like a non-issue since adapter layers are designed to have few parameters (sometimes <1% of the original

model) by having a small bottleneck dimension, which limits the FLOPs they can add. However, large neural networks rely on hardware parallelism to keep the latency low, and adapter layers have to be processed sequentially. This makes a difference in the online inference setting where the batch size is typically as small as one. In a generic scenario without model parallelism, such as running inference on GPT-2 (Radford et al., b) medium on a single GPU, we see a noticeable increase in latency when using adapters, even with a very small bottleneck dimension (Table 1).

Batch Size	center
32	center
16	center
1	center
Sequence Length	center
512	center
256	center
128	center
$\$ \backslash \{\} \Theta \backslash \{\} \$$	center
0.5M	center
11M	center
11M	center
Fine-Tune/LoRA	center
1449.4 ± 0.8	center
338.0 ± 0.6	center
19.8 ± 2.7	center
Adapter $\backslash \{\}^L \backslash \{\} \$$	center
1482.0 ± 1.0 (+2.2%)	center
354.8 ± 0.5 (+5.0%)	center
23.9 ± 2.1 (+20.7%)	center
Adapter $\backslash \{\}^H \backslash \{\} \$$	center
1492.2 ± 1.0 (+3.0%)	center
366.3 ± 0.5 (+8.4%)	center
25.8 ± 2.2 (+30.3%)	center

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over

100 trials. We use an NVIDIA Quadro RTX8000. “ $\$ \backslash \{\} \Theta \backslash \{\} \$$ ” denotes the number of trainable parameters in adapter layers. Adapter $\backslash \{\}^L \backslash \{\} \$$ and Adapter $\backslash \{\}^H \backslash \{\} \$$ are two variants of adapter tuning, which we describe in Section 5.1. The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in Appendix C.

This problem gets worse when we need to shard the model as done in Shoeybi et al. (2020); Lepikhin et al. (2020), because the additional depth requires more synchronous GPU operations such as AllReduce and Broadcast, unless we store the adapter parameters redundantly many times.

Directly Optimizing the Prompt is Hard The other direction, as exemplified by prefix tuning (Li & Liang, 2021), faces a different challenge. We observe that prefix tuning is difficult to optimize and that its performance changes non-monotonically in trainable parameters, confirming similar observations in the original paper. More fundamentally, reserving a part of the sequence length for adaptation necessarily reduces the sequence length available to process a downstream task, which we suspect makes tuning the prompt underperform other methods. We defer the study on task performance to Section 5.

IV. Our Method

We describe the simple design of LoRA and its practical benefits. The principles outlined here apply to any dense layers in deep learning models, though we only focus on certain weights in Transformer language models in our experiments as the motivating use case.

A. Low-Rank-Parametrized Update Matrices

A neural network contains many dense layers which perform matrix multiplication. The weight matrices in these layers typically have full ranks. When adapting to a specific task, Aghajanyan et al. (2020) shows that the pre-trained language models have a low “intrinsic dimension” and can still learn efficiently despite a random projection to a smaller subspace. Inspired by this, we hypothesize the updates to the weights also have a low “intrinsic rank” during adaptation. For a pre-trained weight matrix $W \in \mathbb{R}^{d \times k}$, we constrain its update by representing the latter with a low-rank decomposition $W + \Delta W = W + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$. During training, W is frozen and does not receive gradient updates, while A and B contain trainable parameters. Note both W and $\Delta W = BA$ are multiplied with the same input, and their respective output vectors are summed coordinate-wise. For $h = Wx$, our modified forward pass yields:

$$h = W_0 x + \Delta W x = W_0 x + B A x$$

We illustrate our reparametrization in Figure 1. We use a random Gaussian initialization for A and zero for B , so $\Delta W = BA$ is zero at the beginning of training. We then scale ΔW by $\frac{\alpha}{r}$, where α is a constant in r . When optimizing with Adam, tuning α is roughly the same as tuning the learning rate if we scale the initialization appropriately. As a result, we simply set α to the first r we try and do not tune it. This scaling helps to reduce the need to retune hyperparameters when we vary r (Yang & Hu, 2021).

A Generalization of Full Fine-tuning. A more general form of fine-tuning allows the training of a subset of the pre-trained parameters. LoRA takes a step further and does not require the accumulated gradient update to weight matrices to have full ranks during adaptation. This means that when applying LoRA to all weight matrices and training all biases $\{b_i\}$, we roughly recover the expressiveness of full fine-tuning by setting the LoRA rank r to the rank of the pre-trained weight matrices. In other words, as we increase the number of trainable parameters $\sim r^3$, training LoRA roughly converges to training the original model, while adapter-based methods converge to an MLP and prefix-based methods to a model that cannot take long input sequences.

No Additional Inference Latency. When deployed in production, we can explicitly compute and store $W = W_0 + BA$ and perform inference as usual. Note that both W_0 and BA are in $\mathbb{R}^{d \times k}$. When we need to switch to another downstream task, we can recover W_0 by subtracting BA and then adding a different $B'A'$, a quick operation with very little memory overhead. Critically, this guarantees that we do not introduce any additional latency during inference compared to a fine-tuned model by construction.

B. APPLYING LORA TO TRANSFORMER

In principle, we can apply LoRA to any subset of weight matrices in a neural network to reduce the number of trainable parameters. In the Transformer architecture, there are four weight matrices in

the self-attention module (W_q, W_k, W_v, W_o) and two in the MLP module. We treat W_q (or W_k, W_v) as a single matrix of dimension $d_{\text{model}} \times d_{\text{model}}$, even though the output dimension is usually sliced into attention heads. We limit our study to only adapting the attention weights for downstream tasks and freeze the MLP modules (so they are not trained in downstream tasks) both for simplicity and parameter-efficiency. We further study the effect on adapting different types of attention weight

matrices in a Transformer in Section B.1. We leave the empirical investigation of adapting the MLP layers, LayerNorm layers, and biases to a future work.

Practical Benefits and Limitations. The most significant benefit comes from the reduction in memory and storage usage. For a large Transformer trained with Adam, we reduce that VRAM usage by up to 2/3 if $r \ll d_{\text{model}}$ as we do not need to store the optimizer states for the frozen parameters. On GPT-3 175B, we reduce the VRAM consumption during training from 1.2TB to 350GB. With $r = 4$ and only the query and value projection matrices being adapted, the checkpoint size is reduced by roughly $10,000\times$ (from 350GB to 35MB) $\sim r^4$. This allows us to train with significantly fewer GPUs and avoid I/O bottlenecks. Another benefit is that we can switch between tasks while deployed at a much lower cost by only swapping the LoRA weights as opposed to all the parameters. This allows for the creation of many customized models that can be swapped in and out on the fly on machines that store the pre-trained weights in VRAM. We also observe a 25% speedup during training on GPT-3 175B compared to full fine-tuning $\sim r^5$ as we do not need to calculate the gradient for the vast majority of the parameters. See Appendix L for how to calculate gradient when using LoRA.

LoRA also has its limitations. For example, it is not straightforward to batch inputs to different tasks with different A and B in a single forward pass, if one chooses to absorb A and B into W to eliminate additional inference latency. Though it is possible to not merge the weights and dynamically choose the LoRA modules to use for samples in a batch for scenarios where latency is not critical.

V. EMPIRICAL EXPERIMENTS

We evaluate the downstream task performance of LoRA on RoBERTa (Liu et al., 2019), DeBERTa (He et al., 2021), and GPT-2 (Radford et al., 2019), before scaling up to GPT-3 175B (Brown et al., 2020). Our experiments cover a wide range of tasks, from natural language understanding (NLU) to generation (NLG). Specifically, we evaluate on the GLUE (Wang et al., 2019) benchmark for RoBERTa and DeBERTa. We follow the setup of Li & Liang (2021) on GPT-2 for a direct comparison and add WikiSQL (Zhong et al., 2017) (NL to SQL queries) and SAMSUM (Gliwa et al., 2019) (conversation summarization) for large-scale experiments on GPT-3. See Appendix D for more details on the datasets we use. We use NVIDIA Tesla V100 for all experiments.

A. BASELINES

To compare with other baselines broadly, we replicate the setups used by prior work and reuse

their reported numbers whenever possible. This, however, means that some baselines might only appear in certain experiments. We point out that for most tasks, our goal is not to compete with SOTA because 1) for tasks such as WikiSQL, SOTA uses specialized SQL-aware decoders, and; 2) SOTA makes frequent use of techniques such as ensembling and adversarial training, which might confound our study on adaptation. Thus, we focus on a scenario where we are given a powerful pre-trained autoregressive language model and use as few task-specific modifications as possible. See Appendix E for how we count the number of trainable parameters for different baselines.

Fine-Tuning (FT) is a common approach for adaptation. During fine-tuning, the model is initialized to the pre-trained weights and biases, and all model parameters undergo gradient updates. A simple variant is to update only some layers while freezing others. We include one such baseline reported in prior work (Li & Liang, 2021) on GPT-2, which adapts just the last two layers ($\text{FT}^{\sim\{\text{Top2}\}}$).

>	Model & Method	# Trainable Parameters	MNLI	SST-2	MRPC	CoLA	QNLI	QQP	RTE	STS-B	Avg.
>	RoB{ $\{\}$ text{base}}	(FT)*	125.0M	87.6	94.8	90.2	63.6	92.8	91.9	78.7	91.2

style='text-align:	center;	word-wrap:	
break-word;'	>75.9\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>90.3\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>85.4</td></tr><tr><td>		
style='text-align:	center;	word-wrap:	
break-word;'	>RoB{\{\}text{base}}	(LoRA)</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>0.3M</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>87.5\{\}	
style='text-align:	center;	word-wrap:	
break-word;'	>95.1\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>89.7\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>63.4\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>93.3\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>90.8\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>86.6\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>91.5\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>87.2</td></tr><tr><td>		
style='text-align:	center;	word-wrap:	
break-word;'	>RoB{\{\}text{large}}		
(FT)*</td><td>	style='text-align: center;		
word-wrap:	break-word;'	>355.0M</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>90.2</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>96.4</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>90.9</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>68.0</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>94.7</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>92.2</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>86.6</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>92.4</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>88.9</td></tr><tr><td>		
style='text-align:	center;	word-wrap:	
break-word;'	>RoB{\{\}text{large}}	(LoRA)</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>0.8M</td><td>	style='text-align: center;	
word-wrap:	break-word;'	>90.6\{\}	
style='text-align:	center;	word-wrap:	
break-word;'	>96.2\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>90.9\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>68.2\{\}	</td><td>	
style='text-align:	center;	word-wrap:	
break-word;'	>94.9\{\}	</td><td>	

style='text-align: center;	word-wrap: break-word;'	>96.2\{\}	pm3.0</td><td
style='text-align: center;	word-wrap: break-word;'	>88.7\{\}	pm2.9</td><td
style='text-align: center;	word-wrap: break-word;'	>66.5\{\}	pm4.4</td><td
style='text-align: center;	word-wrap: break-word;'	>94.7\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>92.1\{\}	pm1.0</td><td
style='text-align: center;	word-wrap: break-word;'	>83.4\{\}	pm1.1</td><td
style='text-align: center;	word-wrap: break-word;'	>91.0\{\}	pm1.7</td><td
style='text-align: center;	word-wrap: break-word;'	>87.8</td></tr><tr><td	
style='text-align: center;	word-wrap: break-word;'	>RoB\{\}\text{large}\}	(Adpt [†] {H})†</td><td style='text-align: center;
word-wrap: break-word;'	break-word;'	>0.8M</td><td	
style='text-align: center;	word-wrap: break-word;'	>90.3\{\}	pm3.0</td><td
style='text-align: center;	word-wrap: break-word;'	>96.3\{\}	pm5.0</td><td
style='text-align: center;	word-wrap: break-word;'	>87.7\{\}	pm1.7</td><td
style='text-align: center;	word-wrap: break-word;'	>66.3\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>94.7\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>91.5\{\}	pm1.0</td><td
style='text-align: center;	word-wrap: break-word;'	>72.9\{\}	pm2.9</td><td
style='text-align: center;	word-wrap: break-word;'	>91.5\{\}	pm5.0</td><td
style='text-align: center;	word-wrap: break-word;'	>86.4</td></tr><tr><td	
style='text-align: center;	word-wrap: break-word;'	>RoB\{\}\text{large}\}	(LoRA)†</td><td
style='text-align: center;	word-wrap: break-word;'	>0.8M</td><td style='text-align: center;	
word-wrap: break-word;'	break-word;'	>90.6\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>96.2\{\}	pm5.0</td><td
style='text-align: center;	word-wrap: break-word;'	>90.2\{\}	pm1.0</td><td
style='text-align: center;	word-wrap: break-word;'	>68.2\{\}	pm1.9</td><td
style='text-align: center;	word-wrap: break-word;'	>94.8\{\}	pm3.0</td><td
style='text-align: center;	word-wrap: break-word;'	>91.6\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>85.2\{\}	pm1.1</td><td
style='text-align: center;	word-wrap: break-word;'	>92.3\{\}	pm5.0</td><td
style='text-align: center;	word-wrap: break-word;'		

break-word;'	>88.6</td></tr><tr><td		
style='text-align: center;	word-wrap: break-word;'	>DeB\{\}\text{XXL}\}	
(FT)*</td><td style='text-align: center;			
word-wrap: break-word;'	>1500.0M</td><td		
style='text-align: center;	word-wrap: break-word;'	>91.8</td><td style='text-align: center;	
word-wrap: break-word;'	>97.2</td><td		
style='text-align: center;	word-wrap: break-word;'	>92.0</td><td style='text-align: center;	
word-wrap: break-word;'	>72.0</td><td		
style='text-align: center;	word-wrap: break-word;'	>96.0</td><td style='text-align: center;	
word-wrap: break-word;'	>92.7</td><td		
style='text-align: center;	word-wrap: break-word;'	>93.9</td><td style='text-align: center;	
word-wrap: break-word;'	>92.9</td><td		
style='text-align: center;	word-wrap: break-word;'	>91.1</td></tr><tr><td	
style='text-align: center;	word-wrap: break-word;'	>DeB_{\}\text{XXL}\}	
(LoRA)</td><td style='text-align: center;			
word-wrap: break-word;'	>4.7M</td><td style='text-align: center;		
word-wrap: break-word;'	>91.9\{\}	pm2.0</td><td	
style='text-align: center;	word-wrap: break-word;'	>96.9\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>92.6\{\}	pm6.0</td><td
style='text-align: center;	word-wrap: break-word;'	>72.4\{\}	pm1.1</td><td
style='text-align: center;	word-wrap: break-word;'	>96.0\{\}	pm1.0</td><td
style='text-align: center;	word-wrap: break-word;'	>92.9\{\}	pm1.0</td><td
style='text-align: center;	word-wrap: break-word;'	>94.9\{\}	pm4.0</td><td
style='text-align: center;	word-wrap: break-word;'	>93.0\{\}	pm2.0</td><td
style='text-align: center;	word-wrap: break-word;'	>91.3</td></tr></table>	

<div style="text-align: center;">Table 2: RoBERTa \$ \{base\} \$, RoBERTa \$ \{large\} \$, and DeBERTa \$ \{XXL\} \$ with different adaptation methods on the GLUE benchmark. We report the overall (matched and mismatched) accuracy for MNLI, Matthew’s correlation for CoLA, Pearson correlation for STS-B, and accuracy for other tasks. Higher is better for all metrics. * indicates numbers published in prior works. † indicates runs configured in a setup similar to Houlsby et al. (2019) for a fair comparison.</div>

Bias-only or BitFit is a baseline where we only train the bias vectors while freezing everything else. Contemporarily, this baseline has also been studied by BitFit (Zaken et al., 2021).

Prefix-embedding tuning (PreEmbed) inserts special tokens among the input tokens. These special tokens have trainable word embeddings

and are generally not in the model’s vocabulary. Where to place such tokens can have an impact on performance. We focus on “prefixing”, which prepends such tokens to the prompt, and “infixing”, which appends to the prompt; both are discussed in Li & Liang (2021).

Prefix-layer tuning (PreLayer) is an extension to prefix-embedding tuning. Instead of just learning the word embeddings (or equivalently, the activations after the embedding layer) for some special tokens, we learn the activations after every Transformer layer. The activations computed from previous layers are simply replaced by trainable ones.

Adapter tuning as proposed in Houlsby et al. (2019) inserts adapter layers between the self-attention module (and the MLP module) and the subsequent residual connection. There are two fully connected layers with biases in an adapter layer with a nonlinearity in between. We call this original design \$ Adapter^{\{H\}} \$. Recently, Lin et al. (2020) proposed a more efficient design with the adapter layer applied only after the MLP module and after a LayerNorm. We call it \$ Adapter^{\{L\}} \$. This is very similar to another design proposed in Pfeiffer et al. (2021), which we call \$ Adapter^{\{P\}} \$. We also include another baseline call AdapterDrop (Rücklé et al., 2020) which drops some adapter layers for greater efficiency (\$ Adapter^{\{D\}} \$). We cite numbers from prior works whenever possible to maximize the number of baselines we compare with; they are in rows with an asterisk (*) in the first column.

LoRA adds trainable pairs of rank decomposition matrices in parallel to existing weight matrices. As mentioned in Section 4.2, we only apply LoRA to \$ W_{\{q\}} \$ and \$ W_{\{v\}} \$ in most experiments for simplicity.

B. RoBERTa BASE/LARGE

RoBERTa (Liu et al., 2019) optimized the pre-training recipe originally proposed in BERT (Devlin et al., 2019a) and boosted the latter’s task performance without introducing many more trainable parameters. While RoBERTa has been overtaken by much larger models on NLP leaderboards such as the GLUE benchmark (Wang et al., 2019) in recent years, it remains a competitive and popular pre-trained model for its size among practitioners. We take the pre-trained RoBERTa base (125M) and RoBERTa large (355M) from the HuggingFace Transformers library (Wolf et al., 2020).

<table border=1 style=	margin: auto;
word-wrap: break-word;	><tr><td rowspan=
pan=	2>Model & Method</td><td rowspan=
rowspan=	2># Trainable Parameters</td><td rowspan=
2><td rowspan=	2>BLEU</td><td colspan=
4>E2E NLG Challenge</td></tr><td style=	text-align:

center; word-wrap: break-word;	>NIST</td><td style=
text-align: center; word-wrap: break-word;	>MET</td><td style=
text-align: center; word-wrap: break-word;	>ROUGE-L</td><td style=
text-align: center; word-wrap: break-word;	>CIDEr</td></tr><tr><td style=
text-align: center; word-wrap: break-word;	>GPT-2 M (FT)</td><td style=
text-align: center; word-wrap: break-word;	>354.92M</td><td style=
text-align: center; word-wrap: break-word;	>68.2</td><td style=
text-align: center; word-wrap: break-word;	>8.62</td><td style=
text-align: center; word-wrap: break-word;	>46.2</td><td style=
text-align: center; word-wrap: break-word;	>71.0</td><td style=
text-align: center; word-wrap: break-word;	>2.47</td></tr><tr><td style=
text-align: center; word-wrap: break-word;	>GPT-2 M (Adapter \$ \text{\{}L\}} \$)</td><td style=
text-align: center; word-wrap: break-word;	>0.37M</td><td style=
text-align: center; word-wrap: break-word;	>66.3</td><td style=
text-align: center; word-wrap: break-word;	>8.41</td><td style=
text-align: center; word-wrap: break-word;	>45.0</td><td style=
text-align: center; word-wrap: break-word;	>69.8</td><td style=
text-align: center; word-wrap: break-word;	>2.40</td></tr><tr><td style=
text-align: center; word-wrap: break-word;	>GPT-2 M (Adapter \$ \text{\{}L\}} \$)</td><td style=
text-align: center; word-wrap: break-word;	>11.09M</td><td style=
text-align: center; word-wrap: break-word;	>68.9</td><td style=
text-align: center; word-wrap: break-word;	>8.71</td><td style=
text-align: center; word-wrap: break-word;	>46.1</td><td style=
text-align: center; word-wrap: break-word;	>71.3</td><td style=
text-align: center; word-wrap: break-word;	>2.47</td></tr><tr><td style=
text-align: center; word-wrap: break-word;	>GPT-2 M (Adapter \$ \text{\{}H\}} \$)</td><td style=
text-align: center; word-wrap: break-word;	>11.09M</td><td style=
text-align: center; word-wrap: break-word;	>67.3 \$ \text{\{}pm \$ 6</td><td style=
text-align: center; word-wrap: break-word;	>8.50 \$ \text{\{}pm \$ 0.07</td><td style=
text-align: center; word-wrap: break-word;	>46.0 \$ \text{\{}pm \$ 2</td><td style=
text-align: center; word-wrap: break-word;	>70.7 \$ \text{\{}pm \$ 2</td><td style=
text-align: center; word-wrap: break-word;	>2.44 \$ \text{\{}pm \$ 0.01</td></tr><tr><td style=
text-align: center; word-wrap: break-word;	>GPT-2 M (FT \$ \text{\{}Top2\}} \$)</td><td style=
text-align: center; word-wrap: break-word;	>25.19M</td><td style=

>68.1	>8.59	>46.0	>70.8	>2.41
GPT-2 M (PreLayer)	>0.35M	>69.7	>8.81	>46.1
>2.49	GPT-2 M (LoRA)	>0.35M	>70.4 \$ \{\}pm \$ 1	>8.85 \$ \{\}pm \$ 0.02
>46.8 \$ \{\}pm \$ 2	>71.8 \$ \{\}pm \$ 1	>2.53 \$ \{\}pm \$ 0.02	GPT-2 L (FT)	>774.03M
>68.5	>8.78	>46.0	>69.9	>2.45
GPT-2 L (Adapter \$ \{\}text{L} \$)	>0.88M	>69.1 \$ \{\}pm \$ 1	>8.68 \$ \{\}pm \$ 0.03	>46.3 \$ \{\}pm \$ 0
>71.4 \$ \{\}pm \$ 2	>2.49 \$ \{\}pm \$ 0.0	GPT-2 L (Adapter \$ \{\}text{L} \$)	>23.00M	>68.9 \$ \{\}pm \$ 3
>8.70 \$ \{\}pm \$ 0.04	>46.1 \$ \{\}pm \$ 1	>71.3 \$ \{\}pm \$ 2	>2.45 \$ \{\}pm \$ 0.02	GPT-2 L (PreLayer)*
>0.77M	>70.3	>8.85	>46.2	>71.7
>2.47	GPT-2 L (LoRA)	>0.77M	>70.4 \$ \{\}pm \$ 1	>8.89 \$ \{\}pm \$ 0.02
>46.8 \$ \{\}pm \$ 2	>72.0 \$ \{\}pm \$ 2	>2.47 \$ \{\}pm \$ 0.02		

>23.00M	>68.9 \$ \{\}pm \$ 3	>8.70 \$ \{\}pm \$ 0.04	>46.1 \$ \{\}pm \$ 1	>71.3 \$ \{\}pm \$ 2	>2.45 \$ \{\}pm \$ 0.02
GPT-2 L (PreLayer)*	>0.77M	>70.3	>8.85	>46.2	>71.7
>2.47	GPT-2 L (LoRA)	>0.77M	>70.4 \$ \{\}pm \$ 1	>8.89 \$ \{\}pm \$ 0.02	>46.8 \$ \{\}pm \$ 2
>72.0 \$ \{\}pm \$ 2	>2.47 \$ \{\}pm \$ 0.02				

Table 3: GPT-2 medium (M) and large (L) with different adaptation methods on the E2E NLG Challenge. For all metrics, higher is better. LoRA outperforms several baselines with comparable or fewer trainable parameters. Confidence intervals are shown for experiments we ran. \$ * \$ indicates numbers published in prior works.

and evaluate the performance of different efficient adaptation approaches on tasks from the GLUE benchmark. We also replicate Houlsby et al. (2019) and Pfeiffer et al. (2021) according to their setup. To ensure a fair comparison, we make two crucial changes to how we evaluate LoRA when comparing with adapters. First, we use the same batch size for all tasks and use a sequence length of 128 to match the adapter baselines. Second, we initialize the model to the pre-trained model for MRPC, RTE, and STS-B, not a model already adapted to MNLI like the fine-tuning baseline. Runs following this more restricted setup from Houlsby et al. (2019) are labeled with \$ \{\}dagger \$. The result is presented in Table 2 (Top Three Sections). See Section F.1 for details on the hyperparameters used.

[illegible]

word;	>53.8/29.8/45.9</td></tr><tr><td
style='text-align:	center;
break-word;	>GPT-3 (LoRA)</td><td
style='text-align:	center;
break-word;	>37.7M</td><td
align:	center;
word-wrap:	break-
word;	>74.0</td><td
style='text-align:	center;
word-wrap:	break-word;
style='text-align:	center;
word-wrap:	break-
word;	>53.4/29.2/45.1</td></tr></table>

<div style="text-align: center;">Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAM-Sum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1\%$ for the three metrics.</div>

similar observations in Li & Liang (2021). While a thorough investigation into this phenomenon is out-of-scope for this work, we suspect that having more special tokens causes the input distribution to shift further away from the pre-training data distribution. Separately, we investigate the performance of different adaptation approaches in the low-data regime in Section I.3.

<div style="text-align: center;"></div>

<div style="text-align: center;"></div>

<div style="text-align: center;">Figure 2: GPT-3 175B validation accuracy vs. number of trainable parameters of several adaptation methods on WikiSQL and MNLI-matched. LoRA exhibits better scalability and task performance. See Section I.2 for more details on the plotted data points.</div>

VI. RELATED WORKS

Transformer Language Models. Transformer (Vaswani et al., 2017) is a sequence-to-sequence architecture that makes heavy use of self-attention. Radford et al. (a) applied it to autoregressive

language modeling by using a stack of Transformer decoders. Since then, Transformer-based language models have dominated NLP, achieving the state-of-the-art in many tasks. A new paradigm emerged with BERT (Devlin et al., 2019b) and GPT-2 (Radford et al., b) – both are large Transformer language models trained on a large amount of text – where fine-tuning on task-specific data after pretraining on general domain data provides a significant performance gain compared to training on task-specific data directly. Training larger Transformers generally results in better performance and remains an active research direction. GPT-3 (Brown et al., 2020) is the largest single Transformer language model trained to-date with 175B parameters.

Prompt Engineering and Fine-Tuning. While GPT-3 175B can adapt its behavior with just a few additional training examples, the result depends heavily on the input prompt (Brown et al., 2020). This necessitates an empirical art of composing and formatting the prompt to maximize a model’s performance on a desired task, which is known as prompt engineering or prompt hacking.

Fine-tuning retrains a model pre-trained on general domains to a specific task Devlin et al. (2019b); Radford et al. (a). Variants of it include learning just a subset of the parameters Devlin et al. (2019b); Collobert & Weston (2008), yet practitioners often retrain all of them to maximize the downstream performance. However, the enormity of GPT-3 175B makes it challenging to perform fine-tuning in the usual way due to the large checkpoint it produces and the high hardware barrier to entry since it has the same memory footprint as pre-training.

Parameter-Efficient Adaptation. Many have proposed inserting adapter layers between existing layers in a neural network (Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020). Our method uses a similar bottleneck structure to impose a low-rank constraint on the weight updates. The key functional difference is that our learned weights can be merged with the main weights during inference, thus not introducing any latency, which is not the case for the adapter layers (Section 3). A contemporary extension of adapter is COM-PACTER (Mahabadi et al., 2021), which essentially parametrizes the adapter layers using Kronecker products with some predetermined weight sharing scheme. Similarly, combining LoRA with other tensor product-based methods could potentially improve its parameter efficiency, which we leave to future work. More recently, many proposed optimizing the input word embeddings in lieu of fine-tuning, akin to a continuous and differentiable generalization of prompt engineering (Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). We include comparisons

with Li & Liang (2021) in our experiment section. However, this line of works can only scale up by using more special tokens in the prompt, which take up available sequence length for task tokens when positional embeddings are learned.

Low-Rank Structures in Deep Learning. Low-rank structure is very common in machine learning. A lot of machine learning problems have certain intrinsic low-rank structure (Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013). Moreover, it is known that for many deep learning tasks, a heavily over-parametrized neural network will exhibit low-rank properties after training (Oymak et al., 2019). Some prior works even explicitly impose the low-rank constraint when training the original neural network (Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Khodak et al., 2021; Denil et al., 2014); however, to the best of our knowledge, none of these works considers low-rank update to a frozen model for adaptation to downstream tasks. In theory literature, it is known that neural networks outperform other classical learning methods, including the corresponding (finite-width) neural tangent kernels (Allen-Zhu et al., 2019; Li & Liang, 2018) when the underlying concept class has certain low-rank structure (Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a). Another theory result in Allen-Zhu & Li (2020b) suggests that low-rank adaptations can be useful for adversarial training. In sum, we believe that our proposed low-rank adaptation update is well-motivated by the literature.

VII. CONCLUSION AND FUTURE WORK

Fine-tuning enormous language models is prohibitively expensive in terms of the hardware required and the storage/switching cost for hosting independent instances for different tasks. We propose LoRA, an efficient adaptation strategy that neither introduces inference latency nor reduces input sequence length while retaining high model quality. Importantly, it allows for quick task-switching when deployed as a service by sharing the vast majority of the model parameters. While we focused on Transformer language models, the proposed principles are generally applicable to any neural networks with dense layers.

There are many directions for future works. 1) LoRA can be combined with other efficient adaptation methods, potentially providing orthogonal improvement. 2) The mechanism behind fine-tuning or LoRA is far from clear – how are features learned during pre-training transformed to do well on downstream tasks? We believe that LoRA makes it more tractable to answer this than full fine-tuning. 3) We mostly depend on heuristics to select the weight matrices to apply LoRA to. Are there

more principled ways to do it? 4) Finally, the rank-deficiency of ΔW suggests that W could be rank-deficient as well, which can inspire future works.

References

- [1] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning. arXiv:2012.13255 [cs], December 2020. URL <http://arxiv.org/abs/2012.13255>.
- [2] Zeyuan Allen-Zhu and Yuanzhi Li. What Can ResNet Learn Efficiently, Going Beyond Kernels? In NeurIPS, 2019. Full version available at <http://arxiv.org/abs/1905.10337>.
- [3] Zeyuan Allen-Zhu and Yuanzhi Li. Backward feature correction: How deep learning performs deep learning. arXiv preprint arXiv:2001.04413, 2020a.
- [4] Zeyuan Allen-Zhu and Yuanzhi Li. Feature purification: How adversarial training performs robust deep learning. arXiv preprint arXiv:2005.10190, 2020b.
- [5] Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via overparameterization. In ICML, 2019. Full version available at <http://arxiv.org/abs/1811.03962>.
- [6] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization, 2016.
- [7] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language Models are Few-Shot Learners. arXiv:2005.14165 [cs], July 2020. URL <http://arxiv.org/abs/2005.14165>.
- [8] Jian-Feng Cai, Emmanuel J Candès, and Zuowei Shen. A singular value thresholding algorithm for matrix completion. SIAM Journal on optimization, 20(4):1956–1982, 2010.
- [9] Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017), 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.
- [10] Roman Collobert and Jason Weston. A unified architecture for natural language processing: deep neural networks with multitask learning. In Proceedings of the 25th international conference on Machine learning, ICML ’08, pp. 160–167, New York, NY, USA, July 2008. Association for Computing Machinery. ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.
- [11] Misha Denil, Babak Shakibi, Laurent Dinh, Marc’Aurelio Ranzato, and Nando de Freitas. Predicting parameters in deep learning, 2014.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019a.
- [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805 [cs], May 2019b. URL <http://arxiv.org/abs/1810.04805>. arXiv:1810.04805.
- [14] William B. Dolan and Chris Brockett. Automatically constructing a corpus of sentential paraphrases. In Proceedings of the Third International Workshop on Paraphrasing (IWP2005), 2005. URL <https://aclanthology.org/I05-5002>.

- [15] Claire Gardent, Anastasia Shimorina, Shashi Narayan, and Laura Perez-Beltrachini. The webnlg challenge: Generating text from rdf data. In *Proceedings of the 10th International Conference on Natural Language Generation*, pp. 124–133, 2017.
- [16] Behrooz Ghorbani, Song Mei, Theodor Misiakiewicz, and Andrea Montanari. When do neural networks outperform kernel methods? *arXiv preprint arXiv:2006.13409*, 2020.
- [17] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. *CoRR*, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>.
- [18] Lars Grasedyck, Daniel Kressner, and Christine Tobler. A literature survey of low-rank tensor approximation techniques. *GAMM-Mitteilungen*, 36(1):53–78, 2013.
- [19] Jihun Ham and Daniel D. Lee. Grassmann discriminant analysis: a unifying view on subspace-based learning. In *ICML*, pp. 376–383, 2008. URL <https://doi.org/10.1145/1390156.1390204>.
- [20] Karen Hambardzumyan, Hrant Khachatrian, and Jonathan May. WARP: Word-level Adversarial ReProgramming. *arXiv:2101.00121 [cs]*, December 2020. URL <http://arxiv.org/abs/2101.00121>. *arXiv:2101.00121*.
- [21] Pengcheng He, Xiaodong Liu, Jianfeng Gao, and Weizhu Chen. Deberta: Decoding-enhanced bert with disentangled attention, 2021.
- [22] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-Efficient Transfer Learning for NLP. *arXiv:1902.00751 [cs, stat]*, June 2019. URL <http://arxiv.org/abs/1902.00751>.
- [23] Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. Speeding up convolutional neural networks with low rank expansions. *arXiv preprint arXiv:1405.3866*, 2014.
- [24] Mikhail Khodak, Neil Tenenholz, Lester Mackey, and Nicolò Fusi. Initialization and regularization of factorized neural layers, 2021.
- [25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [26] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.
- [27] Brian Lester, Rami Al-Rfou, and Noah Constant. The Power of Scale for Parameter-Efficient Prompt Tuning. *arXiv:2104.08691 [cs]*, April 2021. URL <http://arxiv.org/abs/2104.08691>. *arXiv:2104.08691*.
- [28] Chunyuan Li, Heerad Farkhor, Rosanne Liu, and Jason Yosinski. Measuring the Intrinsic Dimension of Objective Landscapes. *arXiv:1804.08838 [cs, stat]*, April 2018a. URL <http://arxiv.org/abs/1804.08838>. *arXiv:1804.08838*.
- [29] Xiang Lisa Li and Percy Liang. Prefix-Tuning: Optimizing Continuous Prompts for Generation. *arXiv:2101.00190 [cs]*, January 2021. URL <http://arxiv.org/abs/2101.00190>.
- [30] Yuanzhi Li and Yingyu Liang. Learning overparameterized neural networks via stochastic gradient descent on structured data. In *Advances in Neural Information Processing Systems*, 2018.
- [31] Yuanzhi Li, Yingyu Liang, and Andrej Risteski. Recovery guarantee of weighted low-rank approximation via alternating minimization. In *International Conference on Machine Learning*, pp. 2358–2367. PMLR, 2016.
- [32] Yuanzhi Li, Tengyu Ma, and Hongyang Zhang. Algorithmic regularization in over-parameterized matrix sensing and neural networks with quadratic activations. In *Conference On Learning Theory*, pp. 2–47. PMLR, 2018b.
- [33] Zhaojiang Lin, Andrea Madotto, and Pascale Fung. Exploring versatile generative language model via parameter-efficient transfer learning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 441–459, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.
- [34] Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang, and Jie Tang. GPT Understands, Too. *arXiv:2103.10385 [cs]*, March 2021. URL <http://arxiv.org/abs/2103.10385>. *arXiv:2103.10385*.
- [35] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach, 2019.
- [36] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- [37] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.
- [38] Rabeeh Karimi Mahabadi, James Henderson, and Sebastian Ruder. Compacter: Efficient low-rank hypercomplex adapter layers, 2021.
- [39] Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, et al. Dart: Open-domain structured data record to text generation. *arXiv preprint arXiv:2007.02871*, 2020.
- [40] Jekaterina Novikova, Ondrej Dušek, and Verena Rieser. The e2e dataset: New challenges for end-to-end generation. *arXiv preprint arXiv:1706.09254*, 2017.
- [41] Samet Oymak, Zalan Fabian, Mingchen Li, and Mahdi Soltanolkotabi. Generalization guarantees for neural networks via harnessing the low-rank structure of the jacobian. *arXiv preprint arXiv:1906.05392*, 2019.
- [42] Jonas Pfeiffer, Aishwarya Kamath, Andreas Rücklé, Kyunghyun Cho, and Iryna Gurevych. Adapter-fusion: Non-destructive task composition for transfer learning, 2021.
- [43] Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, and Sanjeev Khudanpur. Semi-orthogonal low-rank matrix factorization for deep neural networks. In *Interspeech*, pp. 3743–3747, 2018.
- [44] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving Language Understanding by Generative Pre-Training. pp. 12, a.
- [45] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language Models are Unsupervised Multitask Learners. pp. 24, b.
- [46] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. *CoRR*, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>.
- [47] Sylvestre-Alvise Rebuffi, Hakan Bilen, and Andrea Vedaldi. Learning multiple visual domains with residual adapters. *arXiv:1705.08045 [cs, stat]*, November 2017. URL <http://arxiv.org/abs/1705.08045>. *arXiv:1705.08045*.
- [48] Andreas Rücklé, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers, and Iryna Gurevych. Adapterdrop: On the efficiency of adapters in transformers, 2020.
- [49] Tara N Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, and Bhuvana Ramabhadran. Low-rank matrix factorization for deep neural network training with high-dimensional output targets. In *2013 IEEE international conference on acoustics, speech and signal processing*, pp. 6655–6659. IEEE, 2013.
- [50] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism, 2020.
- [51] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*,

- pp. 1631–1642, Seattle, Washington, USA, October 2013. Association for Computational Linguistics. URL <https://aclanthology.org/D13-1170>.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, pp. 6000–6010, 2017.
 - [53] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding, 2019.
 - [54] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. Superglue: A stickier benchmark for general-purpose language understanding systems, 2020.
 - [55] Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*, 2018.
 - [56] Adina Williams, Nikita Nangia, and Samuel Bowman. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 1112–1122, New Orleans, Louisiana, June 2018. Association for Computational Linguistics. doi: 10.18653/v1/N18-1101. URL <https://www.aclweb.org/anthology/N18-1101>.
 - [57] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
 - [58] Greg Yang and Edward J. Hu. Feature Learning in Infinite-Width Neural Networks. *arXiv:2011.14522 [cond-mat]*, May 2021. URL <http://arxiv.org/abs/2011.14522>. *arXiv:2011.14522*.
 - [59] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models, 2021.
 - [60] Yu Zhang, Ekapol Chuangsuwanich, and James Glass. Extracting deep neural network bottleneck features using low-rank matrix factorization. In *2014 IEEE international conference on acoustics, speech and signal processing (ICASSP)*, pp. 185–189. IEEE, 2014.
 - [61] Yong Zhao, Jinyu Li, and Yifan Gong. Low-rank plus diagonal adaptation for deep neural networks. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5005–5009. IEEE, 2016.
 - [62] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103, 2017. URL <http://arxiv.org/abs/1709.00103>.